

The Visual Studio .Net

- "The Microsoft visual studio.net is a complete software package that is use to developdifferent Software and application".
- Visual studio is a product of The Microsoft Company.
- It is also known as web development tool.
- Microsoft visual studio.net has an **integrated development environment**.
- Visual studio.net supports various languages. Mostly supported languages are as followlike: **Visual basic.net, Visual c + +.net, c#.net, j#.net etc.**

Some of Applications which can be developed using visual stdio.net

- Some common type of project can be developed using visual studio.net.
- Asp.net based applications and dynamic websites.
- DOS base / Console applications.
- Window / desktop applications.
- Smart devices (Mobile device, PDA [Personal Digital Assistant]) applications.
- Class libraries (Three tier business logic, Namespaces).
- Custom controls (Web or windows).
- Web services (HTTP, XML [Extensible Markup Language] and SO [Simple Object]).

Versions of Microsoft Visual studio.net

Visual studio.net versions can be given as below with its default .net framework.

Sr. No.	Visual studio.net versions	Default .net framework
1	Visual studio.net	1.0
2	Visual studio.net 2003	1.1
3	Visual studio.net 2005	2.0
4	-	3.0
5	Visual studio.net 2008	3.5
6	Visual studio.net 2010	4.0
7	Visual studio.net 2012	4.5
8	Visual studio.net 2013	4.5.1
9	Visual studio.net 2015	4.6

Features of .Net Framework

Microsoft visual studio.net has many support features and functionalities. Those facilities are detailed as below:

Interoperability

Because computer systems commonly require interaction between newer and older applications,

.NET Framework provides means to access functionality implemented in newer and older programs that execute outside .NET environment. Access to COM components is provided in System.Runtime.InteropServices and System.EnterpriseServices namespaces of the framework; access to other functionality is achieved using the P/Invoke feature.

Common Language Runtime engine

Common Language Runtime (CLR) serves as the execution engine of .NET Framework. All

.NET programs execute under the supervision of CLR, guaranteeing certain properties and behaviors in the areas of memory management, security, and exception handling.

Language independence

.NET Framework introduces a Common Type System, or CTS. CTS specification defines all possible data types and programming constructs supported by CLR and how they may or may not interact with each other conforming to Common Language Infrastructure (CLI) specification. Because of this feature, .NET Framework supports the exchange of types and object instances between libraries and applications written using any conforming .NET language.

Framework Class Library

Framework Class Library (FCL) is a library of functionality available to all languages using .NET Framework. FCL provides classes that encapsulate a number of common functions, including file reading and writing, graphic rendering, database interaction, XML document manipulation, and so on. It consists of classes, interfaces of reusable types that integrates CLR.

Simplified deployment

.NET Framework includes design features and tools which help manage the installation of computer software to ensure that it does not interfere with previously installed software, and that it conforms to security requirements.

Security

The design addresses some of the vulnerabilities, such as buffer overflows, which have been exploited by malicious software. Additionally, .NET provides a common security model for all applications.

Portability

Cross-platform implementations are available for other operating systems. This makes it possible for third parties to create compatible implementations of the framework and its languages on other platforms.

Rich set of class

Microsoft visual studio.net Contain number of **classes & namespaces** that provides variety of functionalities in our program. Classes are like header files in C and C++ programming languages. Ex:- If we want to work with database we need to include "System .data" namespace.

Object oriented programming system

Microsoft visual studio support and provide fully object oriented programming environment. Ex:-C # is a fully object oriented programming language which is supported by .net.

Automatic memory management

The .Net Support automatic memory management so that the developers not need to worry for system resources like Memory free up with garbage collector. The garbage collector take responsibility for free up release unused objects.

Multi language support

Microsoft visual studio.net support multiple languages like; c#, j#, vb.net, c ++.net etc. It is also known as inter ability this means that if your friend skill in c ++ and you well in java

so both can use different skills in develop application using visual studio.net. Each language is different but the basic an environment of developing are same.

Multi device support

Microsoft visual studio.net support many multiple devices. Ex.:-We have to create and develop mobile applications and PDAs supported software.

Easy and fast web development

Asp.net is very useful for developing dynamic or static web application or website.

Web application - Using through internet but particular private network use for to workin application.

Website - Asp.net contains rich controls for web development.

XML support

XML stands for Extensible Markup Language. Visual studio.net support for writing (runtime), manipulating, and transmitting (Transfer services to one webpage to another webpage).

Ease of development & configuration

Developing window applications those use "COM"-Component Object Model. .net does not need to any registration such as much of development is simplified.

*****.net architecture**

The .NET architecture for advanced web technology revolves around several key

components and layers that **work together to provide a robust, scalable, and**

maintainable environment for web applications. Here, we'll explore the main components and how they fit together, accompanied by a diagram for clarity.

Key Components of .NET Architecture

1. Presentation Layer:

- Handles the UI and user interaction.

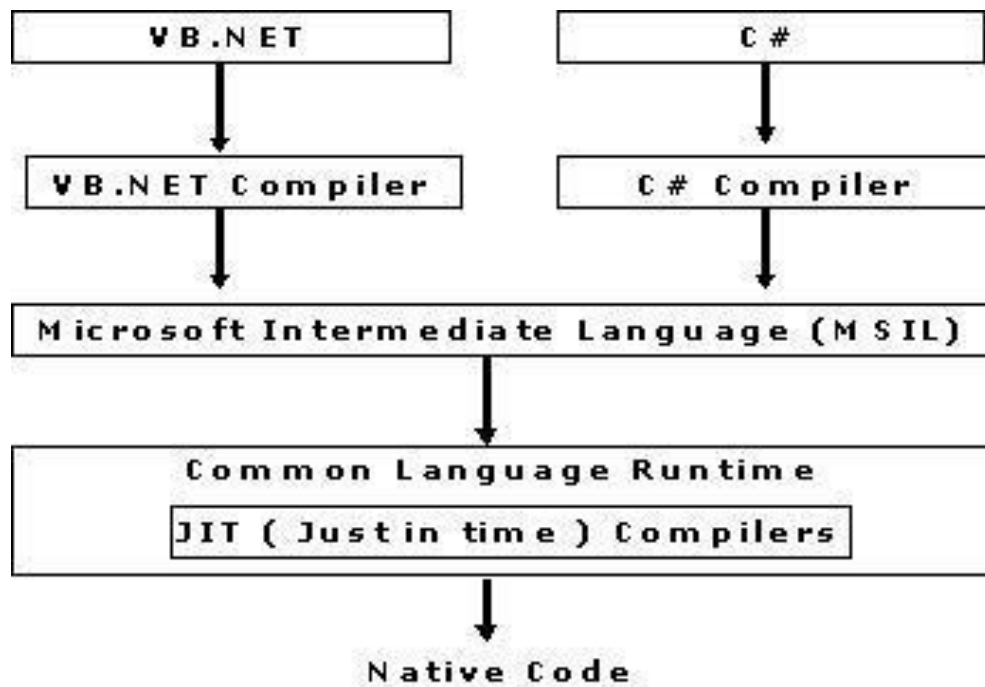
- Various frameworks like ASP.NET MVC[**Model-View-Controller**], Razor Pages, Blazor, etc., provide flexibility in building the front-end.
- 2. **Business Logic Layer:**
 - Encapsulates the core functionality and business rules.
 - Ensures that the application logic is separate from data access and presentation concerns.
- 3. **Data Access Layer:**
 - Manages data retrieval and storage.
 - ORMs like Entity Framework streamline data operations, while ADO.NET and Dapper provide more control for complex scenarios.
- 4. **Service Layer:**
 - Exposes functionalities as services for external or internal consumption.
 - Supports RESTful API[**Application Programming Interface**.]s, RPCs [**Remote Procedure Call**] via gRPC, and legacy services with WCF[**Windows Communication Foundation**].
- 5. **Database Layer:**
 - The **backend** storage, where data resides.
 - Supports various relational and NoSQL databases.
- 6. **Infrastructure Layer:**
 - Deals with the operational aspects such as logging, caching, and security.
 - Ensures that these cross-cutting concerns are managed consistently across the application.
- 7. **Identity and Security:**
 - Manages authentication and authorization.
 - Implements security protocols to safeguard application access and data integrity.

This architecture provides a comprehensive framework for developing modern, scalable, and secure web applications using .NET technologies.

*****CLR[Common Language Run Time]**

- >.NET CLR is a **runtime environment** that manages and executes the code written in any .NET Programming language.
- >CLR is the **virtual machine component** of the .NET framework.
- >That language's compiler compiles the source code of applications developed using .NET compliant languages into CLR's intermediate language called **MSIL[Microsoft Intermediate Language]**

.NET CLR Structure



*****The main components of CLR are:**

- Common type system [CTS]
- Common language specification [CLS]
- Garbage Collector [GC]
- Just in Time Compiler[JIT]
- Metadata and Assemblies

*****CLS [COMMON LANGUAGE SPECIFICATION]**

It is a sub set of CTS and it specifies a set of rules that needs to be adhered or satisfied by all language compilers targeting CLR. It helps in cross language inheritance and cross language debugging.

Common language specification Rules:

It describes the minimal and complete set of features to produce code that can be hosted by CLR. It ensures that products of compilers will work properly in .NET environment.

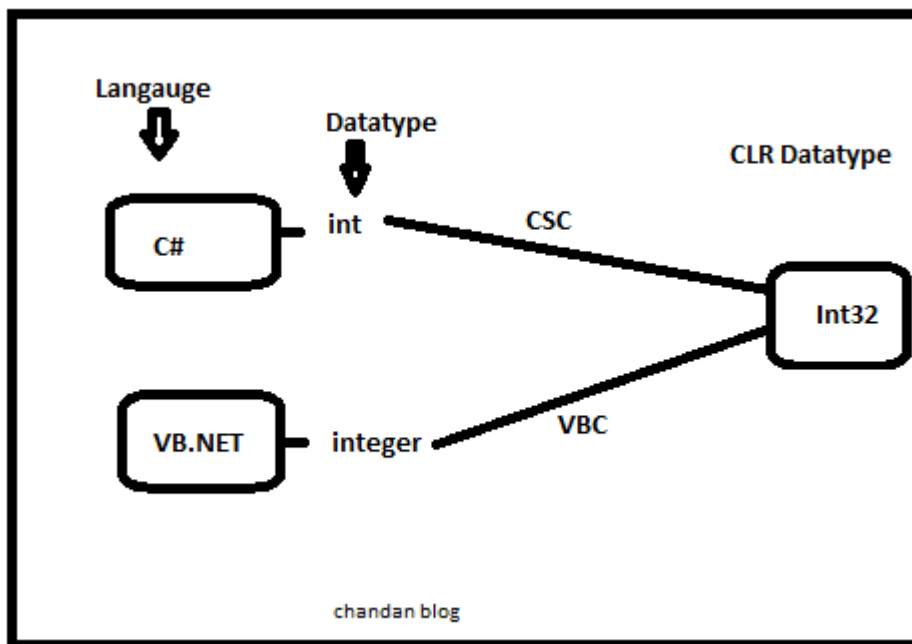
Sample Rules:

1. Representation of text strings
2. Internal representation of enumerations

3. Definition of static members and this is a subset of the CTS which all .NET languages are expected to support.
4. Microsoft has defined CLS which are nothing but guidelines that language to follow so that it can communicate with other .NET languages in a seamless manner.

*****CTS [COMMON TYPE SYSTEM]**

CTS provides guidelines for **declaring, using, and managing data types at runtime**. It offers cross-language communication. For example, VB.NET has an integer data type, and C# has an int data type for managing integers. After compilation, Int32 is used by both data types. So, CTS provides the data types using managed code. A common type system helps in writing language-independent code.



It provides two categories of Types:-

1. **Value Type:** A value type stores the data **in memory allocated** on the stack or inline in a structure. This category of Type holds the data directory. If one variable's value is copied to another, both the variables store data independently. It can be of inbuilt-in types, user-defined, or enumerations types. Built-in types are primitive data types like numeric, Boolean, char, and date. Users in the source code create user-defined types. An enumeration refers to a set of enumerated values

represented by labels but stored as a numeric type.

2. **Reference Type:** A Reference type stores a reference to the value of a memory address and is allocated on the heap. Heap memory is used for dynamic memory allocation. Reference Type does not hold actual data directly but holds the address of data. Whenever a reference type object is made, it copies the address and not actual data. Therefore two variables will refer to the same data. If data of one Reference Type object is changed, the same is reflected for the other object. Reference types can be self-describing types, pointer types, or interference types. The self-describing types may be string, array, and class types that store metadata about themselves.

***FCL, [FRAMEWORK CLASS LIBRARY]

. This is also called as Base Class Library and it is common for all types of applications i.e. the way you access the Library Classes and Methods in VB.NET will be the same in C#, and it is common for all other languages in .NET.

The following are different types of applications that can make use of .net class library.

1. Windows Application.
2. Console Application
3. Web Application.
4. XML Web Services.
5. Windows Services.

The main namespaces in the .NET class library

Namespace	Description
System	Contains fundamental classes and base classes that define common value and reference data types, events and event handlers, interfaces, processing exceptions, data conversion, mathematics, as well as garbage collection and application environment management. Some of the classes defined in the namespace are covered in this chapter and also in Chapters 4 and 5.
System.IO	Provides classes that support asynchronous and synchronous reading from and writing to data streams and files. Contains classes like FileStream, MemoryStream, Path, and Directory. We will learn about this namespace and its classes in the next chapter.

Namespace	Description
System.Collections	Contains interfaces and classes that define various collections of objects, such as lists, arrays, hash tables, stacks, queues, and dictionaries. The classes defined in this namespace are covered in this chapter.
System.Threading	Provides classes and interfaces that support multithreaded programming. Contains classes such as Thread, ThreadPool, Mutex, and AutoResetEvent.

*****MSIL OR CIL [Microsoft Intermediate Language or Common Intermediate Language]**

Microsoft Intermediate Language (MSIL) is a low-level programming language used by the .NET framework. It is the CPU-independent instruction set into which .NET source code is compiled. Understanding MSIL can provide insights into the execution of ASP.NET applications and the .NET runtime in general.

```

| Source Code |
+-----+
| Language   |
+-----+
| Compiler   |
+-----+
| MSIL (CIL) |
+-----+
| ILDasm (Optional) |
+-----+
| JIT Compiler |
+-----+
| Native Code |
+-----+
| Execution   |
+-----+

```

Key Concepts of MSIL in ASP.NET

1. Compilation Process:

- When you compile an ASP.NET application (or any .NET application), the source code written in high-level languages such as C# or VB.NET is compiled into MSIL.
- This MSIL code is stored in assemblies (DLL or EXE files).

2. Execution Process:

- At runtime, the .NET Common Language Runtime (CLR) Just-In-Time (JIT) compiles the MSIL into native machine code specific to the architecture of the target machine.
- This JIT compilation happens method-by-method as each method is called for the first time, improving startup performance.

3. **Portability:**

- Since MSIL is CPU-independent, .NET applications can be executed on any platform with a compatible .NET runtime. This is a key feature that allows .NET applications to be cross-platform.

4. **Security and Verification:**

- MSIL includes metadata and type information that allow the CLR to enforce security and type safety.
- The CLR verifies the MSIL before executing it to ensure that it meets the required safety and security constraints.

5. **Reflection:**

- MSIL allows for reflection, enabling runtime inspection and modification of code. This is often used for dynamic type discovery and late binding.

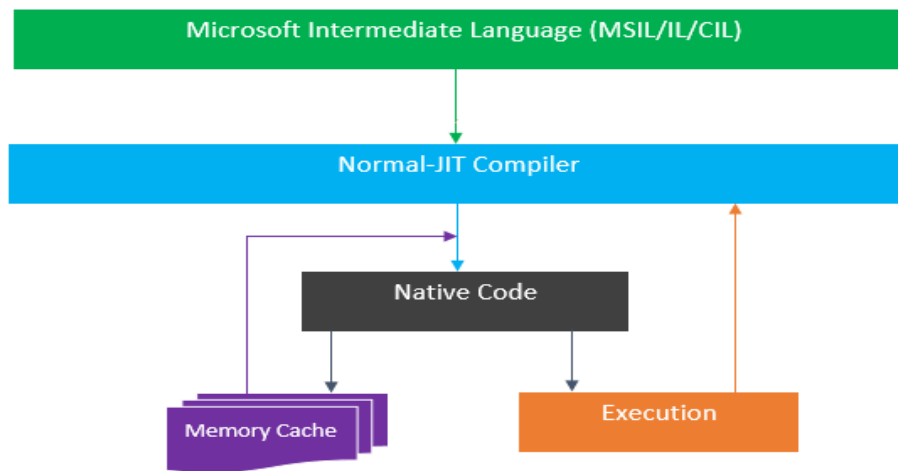
*****JIT,**

Just in Time [JIT] is a smart compiler because it does not compile whole program code each and every code, It compile only that portion of the program's functions. The .Net languages , which is conforms to the Common Language Specification (CLS), uses its corresponding runtime to run the application on different Operating Systems . During the code execution time, the Managed Code compiled only when it is needed, that is it converts the appropriate instructions to thenative code for execution just before when each function is called. This process is called Just In

Types of JIT

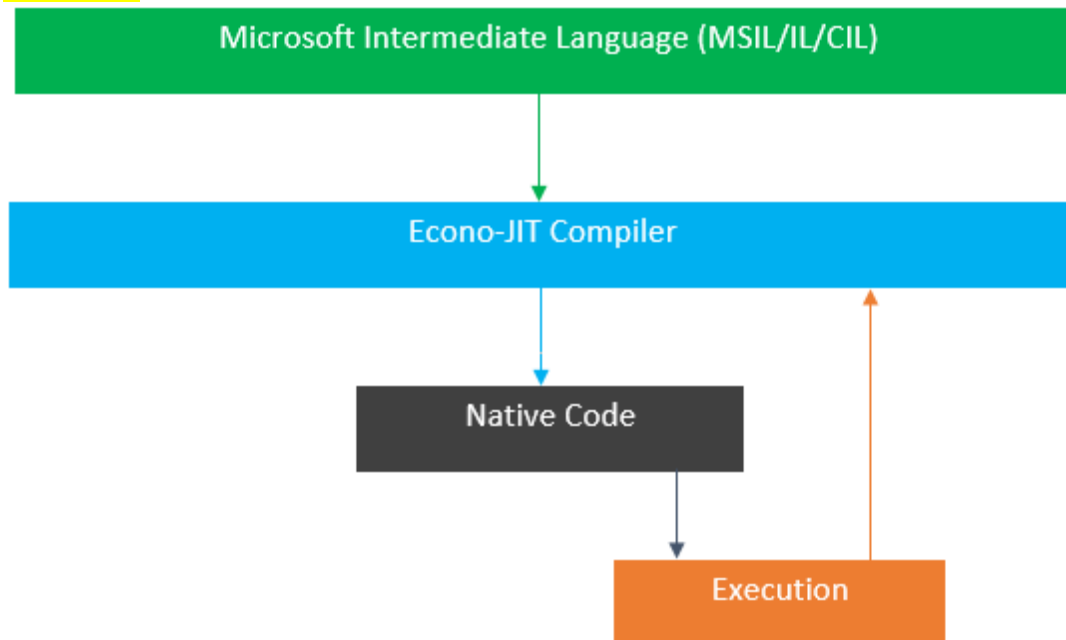
Normal JIT

This compiles only those methods that are called at runtime. These methods are compiled only first time when they are called, and then they are stored in memory cache. This memory cache is commonly called as JITTED. When the same methods are called again, the compiled code from cache is used for execution.



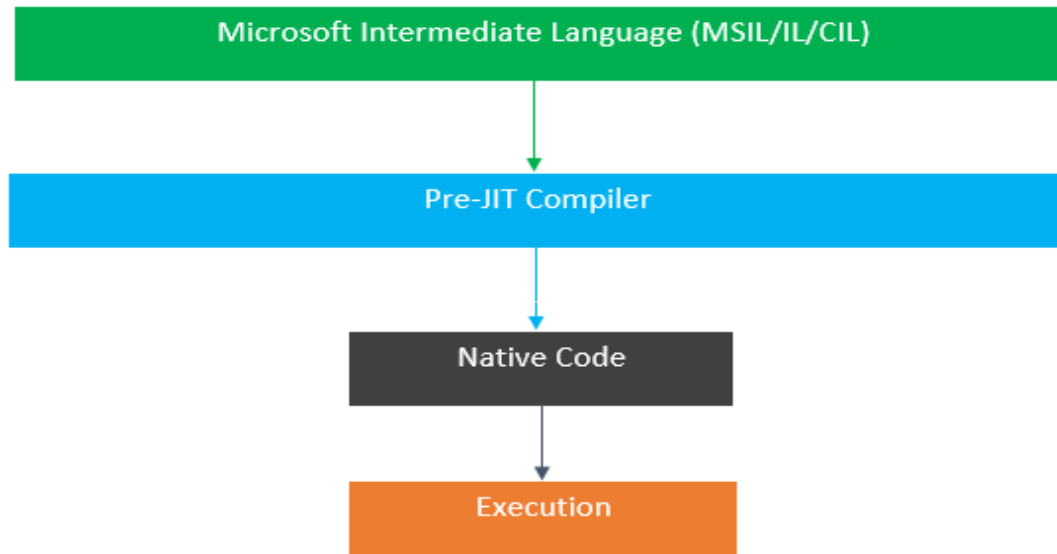
Econo JIT

This compiles only those methods that are called at runtime and removes them from memory after execution



Pre JIT

This compiles entire MSIL code into native code in a single compilation cycle. This is done at the time of deployment of the application



*****WPF [Windows Presentation Foundation]**

WPF, which stands for Windows Presentation Foundation, is Microsoft's latest approach to a GUI framework, used with the .NET framework.

A GUI framework allows you to create an application with a wide range of GUI elements, like labels, textboxes and other well known elements.

There are a lot of GUI frameworks out there, but for .NET developers, the most interesting ones are currently WinForms and WPF. WPF is the newest, but Microsoft is still maintaining and supporting WinForms. As you will see in the next chapter, there are quite a few differences between the two frameworks, but their purpose is the same: To make it easy to create applications with a great GUI.

WPF advantages

- It's newer and thereby more in tune with current standards
- Microsoft is using it for a lot of new applications, e.g. Visual Studio
- It's more flexible, so you can do more things without having to write or buy new controls
- Databinding, which allows you to get a more clean separation of data and layout
- Uses hardware acceleration for drawing the GUI, for better performance
- It allows you to make user interfaces for both Windows applications and web applications (Silverlight/XBAP)

*****WCF [Windows Communication Foundation],**

Windows Communication Foundation (WCF) is a **framework for building service-oriented applications. With WCF, you can send data as asynchronous messages from one service endpoint to another.** A service endpoint can be part of a continuously available service hosted by IIS, or it can be a service hosted in an application.

the key concepts and components involved in using WCF in ASP.NET:**1. Service Contract**

A service contract defines the operations that a service provides. It is defined using an interface and decorated with the [ServiceContract] attribute, and the operations are decorated with the [OperationContract] attribute.

2. Data Contract

A data contract defines the data types that are exchanged between the client and the service. Data contracts are defined using classes and are decorated with the [DataContract] attribute. The members of the class that need to be serialized are decorated with the [DataMember] attribute.

3. Endpoints

Endpoints specify the address where the service can be found, the binding that is used to communicate with the service, and the contract that describes the operations available. An endpoint has three components:

- **Address:** Specifies the location where messages are sent.
- **Binding:** Defines how messages are sent.
- **Contract:** Specifies what operations the endpoint exposes.

4. Hosting

A WCF service must be hosted in a Windows process. The service can be hosted in various environments, such as:

- **IIS (Internet Information Services):** Suitable for HTTP-based services.
- **Self-hosting:** Hosting the service in a managed application like a console app, Windows service, etc.
- **Windows Activation Services (WAS):** A component of IIS that allows hosting services over different protocols.

5. Bindings

Bindings define how the client and service communicate with each other. WCF provides several types of bindings:

- **BasicHttpBinding:** Used for simple HTTP communication.
- **WSHttpBinding:** Provides security, reliability, and other WS-* standards.
- **NetTcpBinding:** Used for communication over TCP.
- **NetNamedPipeBinding:** Used for communication between WCF services on the same machine.
- **NetMsmqBinding:** Used for communication with queued messages.

6. Behaviors

Behaviors are components that modify or extend the service or client. They are used to manage runtime properties, such as security, instance management, transaction management, etc.

7. Metadata

WCF can expose metadata that describes the service, which can be consumed by clients to generate proxy classes. This metadata is usually available in the form of WSDL (Web Services Description Language) documents.

8. Configuration

WCF services can be configured using configuration files (web.config or app.config) or through code. The configuration includes defining the service, bindings, behaviors, and endpoints.

9. Security

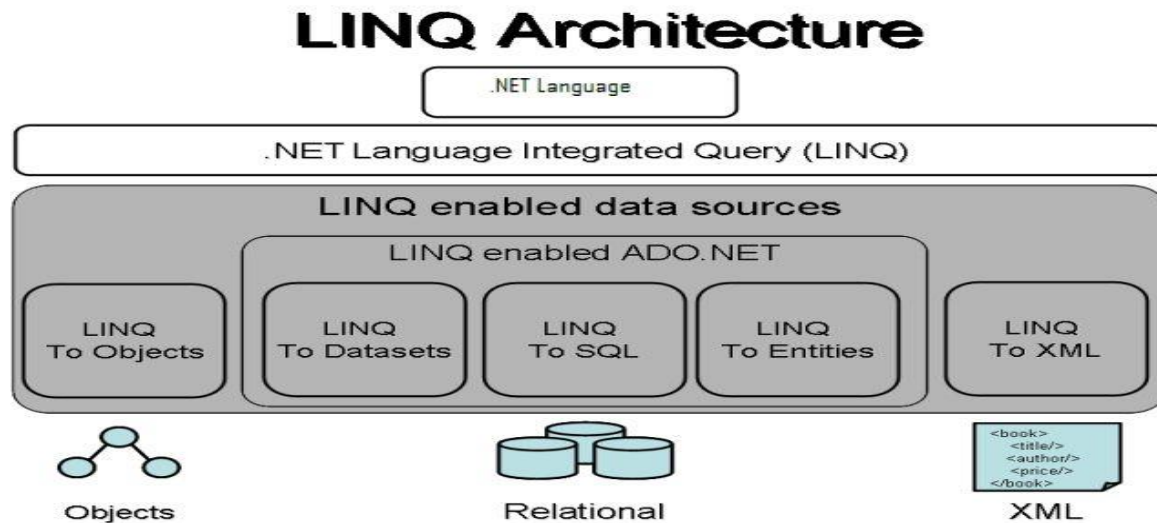
WCF provides various security mechanisms to secure the communication between the client and service. Security can be at the message level or transport level and includes authentication, authorization, confidentiality, and integrity.

Advantages of WCF

- Implementing the security model and binding change in WCF do not require a major change in coding. Just a few configuration changes is required to meet the constraints.
- WCF has built-in logging mechanism whereas in other technologies, it is essential to do the requisite coding.
- WCF has integrated AJAX and support for JSON (JavaScript object notation).
- It offers scalability and support for up-coming web service standards.
- It has a default security mechanism which is extremely robust.

*****LINQ [Language-Integrated Query]**

Language-Integrated Query (LINQ) is a powerful set of technologies based on the integration of query capabilities directly into the C# language. LINQ Queries are the first-class language construct in C# .NET, just like classes, methods, events. The LINQ provides a consistent query experience to query objects (LINQ to Objects), relational databases (LINQ to SQL), and XML (LINQ to XML).



LINQ queries return results as objects. It enables you to use object-oriented approach on the result set and not to worry about transforming different formats of results into objects.



LINQ (Language Integrated Query) is uniform query syntax in C# and VB.NET to retrieve data from different sources and formats. It is integrated in C# or VB, thereby eliminating the mismatch between programming languages and databases, as well as providing a single querying interface for different types of data sources.

Uses of LINQ

1. It additionally gives full sort checking at the compile time. It assists us with distinguishing the error at the runtime, so we can undoubtedly eliminate them.
2. LINQ is it is straightforward, very much arranged, and significant level language than SQL

3. You can likewise utilize LINQ with C# array and collections. It provides you with another guidance to take care of the old issues in a powerful way.
4. With the assistance of LINQ, you can undoubtedly work with a data source like XML, SQL, Entities, objects, and so forth. A single query can work with the database, compelling reason need to learn various sorts of languages.
5. LINQ upholds query expression, Anonymous types, Implicitly typed variables, Lambda expressions, Object and collection initializers, and Extension methods.

****.Net Languages.

.NET supports multiple programming languages, which allows developers to use the language they are most comfortable with or the one most suitable for the task at hand. Here are some of the primary languages used in .NET development:

1. C# (C Sharp)

- **Overview:** C# is the most commonly used language in the .NET ecosystem. It is a modern, object-oriented language designed for simplicity, readability, and type safety.
- **Use Cases:** Web applications, desktop applications, mobile apps (Xamarin), game development (Unity), and cloud services.
- **Key Features:** Strong typing, garbage collection, language-integrated query (LINQ), asynchronous programming (async/await), pattern matching, and interoperability with other .NET languages.

2. VB.NET (Visual Basic .NET)

- **Overview:** VB.NET is an evolution of the classic Visual Basic language. It is designed to be easy to learn and use, especially for those who have experience with older versions of Visual Basic.
- **Use Cases:** Rapid application development (RAD), legacy application maintenance, and enterprise applications.
- **Key Features:** Ease of use, strong integration with the .NET Framework, and robust support for COM interoperability.

3. F#

- **Overview:** F# is a functional-first programming language that also supports object-oriented and imperative programming. It is known for its concise syntax and powerful features for data manipulation.
- **Use Cases:** Data science, machine learning, financial modeling, and applications requiring complex mathematical computations.
- **Key Features:** Immutability by default, pattern matching, type inference, and support for both functional and object-oriented paradigms.

4. ASP.NET

- **Overview:** ASP.NET is not a language but a framework for building web applications and services with .NET. It supports multiple languages, primarily C# and VB.NET.
- **Use Cases:** Web applications, web APIs, and real-time web apps.
- **Key Features:** Razor syntax for dynamic web pages, SignalR for real-time communication, and comprehensive security features.

5. PowerShell

- **Overview:** PowerShell is a task automation and configuration management framework consisting of a command-line shell and scripting language. It is built on the .NET Framework.
- **Use Cases:** System administration, automation of repetitive tasks, and configuration management.
- **Key Features:** Command-line interface, scripting capabilities, and deep integration with Windows and other Microsoft products.

6. XAML (Extensible Application Markup Language)

- **Overview:** XAML is a declarative XML-based language used for initializing structured values and objects. It is primarily used for designing user interfaces in .NET applications.
- **Use Cases:** User interface design for WPF (Windows Presentation Foundation), UWP (Universal Windows Platform), and Xamarin.Forms.
- **Key Features:** Declarative syntax, data binding, and support for creating complex UI layouts.

***Introduction to C#:-

C# is pronounced as "C-Sharp". It is an object-oriented programming language provided by Microsoft that runs on .Net Framework.

By the help of C# programming language, we can develop different types of secured and robust applications:

- Window applications
- Web applications
- Distributed applications
- Web service applications
- Database applications etc.

C# is approved as a standard by ECMA and ISO. C# is designed for **CLI (Common Language Infrastructure)**. CLI is a specification that describes executable code and runtime environment.

******Data type**

- Data types are play most importance role in any programming language. Data types are used in all programming language.
- A Variable must be declared with the data type. We need to inform the compiler about which types of data should be stored in a variable at the time of variable declaration.
- We will learn about variable in next asp.net tutorials. In variable which types of data can be store it depend on data types of variable.

```
protected void Page_Load(object sender, EventArgs e)
{
    int intvariable = 56;

    string strvariable = "Meera Academy";

    char charvariable = 'M';

    float floatvariable = 56.2F;

    double doublevariable = 123.56;

    bool boolvariable = true;
}
```

Integer Data types in C#

For integer we use **int** in C#. The data should be **number or digit**. When we are working with digit or number the **int** is most commonly used data types. All data types have it's range to store data.

```
int a=10;
```

`int a = 10;` ——— Value of Variable
 ↙ ↘
 Data type Variable Name

int data types in c# .net

String Data types in C#.

String is declared as small **string** in C#. The string data types store the **text value, a number of character**. String variable store data with in **double quote**.

```
string strname="Hello Meera";
```

```
string strname = "Hello Meera";
```

Data Type
Variable Name
Value

string data types in c#.net

Char data types in C#.

Char are use to store a **single character**. We can store more character if we assign size while declaring char variable.

```
char abc = 'A';
```

Float and double data types in C#.

- Float is a **32-bit** digit number.
- Float can hold up to **7 digit number**.
- Double is a **64-bit** digit number.
- Double can have up to **16 digit number**.

Float and double data types are used to store integer number which may or may not contain decimals.

```
float a = 12.5F;
```

```
double db = 1235.6;
```

Bool data types in C#.

Bool data types have two values either True or False.

Data types are classified as value type or reference type. It depending on a variable store its on data or a pointer to the data in memory.

***Expressions

ASP.NET expressions are a declarative way **set control properties based on information** that is evaluated at run time. For example, you can use expressions to set a property to values that are based on connection strings, application settings, and other values contained within an application's configuration and resource files.

Arithmetic Expressions

- **Add:** Represents an addition operation.

```

using System;
using System.Linq.Expressions;

class Program
{
    static void Main()
    {
        var left = Expression.Constant(5);
        var right = Expression.Constant(3);
        var add = Expression.Add(left, right);
        var addLambda = Expression.Lambda<Func<int>>(add).Compile();
        Console.WriteLine(addLambda());

        // Pause the console to see the output
        Console.WriteLine("Press any key to exit...");
        Console.ReadKey();
    }
}
//

```

Output: 8

Subtract: Represents a subtraction operation.

```

var subtract = Expression.Subtract(left, right);
var subtractLambda = Expression.Lambda<Func<int>>(subtract).Compile();
Console.WriteLine(subtractLambda()); // Output: 2

```

- **Multiply:** Represents a multiplication operation.

```

var multiply = Expression.Multiply(left, right);
var multiplyLambda = Expression.Lambda<Func<int>>(multiply).Compile();
Console.WriteLine(multiplyLambda()); // Output: 15

```

- **Divide:** Represents a division operation.

```

var divide = Expression.Divide(left, right);
var divideLambda = Expression.Lambda<Func<int>>(divide).Compile();
Console.WriteLine(divideLambda()); // Output: 1

```

Logical Expressions

- **And:** Represents a bitwise AND operation.

```

var leftBool = Expression.Constant(true);
var rightBool = Expression.Constant(false);
var and = Expression.And(leftBool, rightBool);
var andLambda = Expression.Lambda<Func<bool>>(and).Compile();
Console.WriteLine(andLambda()); // Output: False

```

- **Or:** Represents a bitwise OR operation.

```

var or = Expression.Or(leftBool, rightBool);
var orLambda = Expression.Lambda<Func<bool>>(or).Compile();
Console.WriteLine(orLambda()); // Output: True

```

Comparison Expressions

- **Equal:** Represents an equality comparison.

```
var equal = Expression.Equal(left, right);
var equalLambda = Expression.Lambda<Func<bool>>(equal).Compile();
Console.WriteLine(equalLambda()); // Output: False
```

- **GreaterThan:** Represents a greater-than comparison.

```
var greaterThan = Expression.GreaterThan(left, right);
var greaterThanLambda = Expression.Lambda<Func<bool>>(greaterThan).Compile();
Console.WriteLine(greaterThanLambda()); // Output: True
```

Conditional Expressions

- **Conditional:** Represents a conditional (ternary) operation.

```
var condition = Expression.Constant(true);
var ifTrue = Expression.Constant(1);
var ifFalse = Expression.Constant(0);
var conditional = Expression.Condition(condition, ifTrue, ifFalse);
var conditionalLambda = Expression.Lambda<Func<int>>(conditional).Compile();
Console.WriteLine(conditionalLambda()); // Output: 1
```

Constant and Parameter Expressions

- **Constant:** Represents a constant value.

```
var constant = Expression.Constant(42);
var constantLambda = Expression.Lambda<Func<int>>(constant).Compile();
Console.WriteLine(constantLambda()); // Output: 42
```

- **Parameter:** Represents a named parameter.

```
var param = Expression.Parameter(typeof(int), "x");
var paramLambda = Expression.Lambda<Func<int, int>>(param, param).Compile();
Console.WriteLine(paramLambda(5)); // Output: 5
```

Method Call Expressions

- **Call:** Represents a method call.

```
var method = typeof(Console).GetMethod("WriteLine", new Type[] { typeof(string) });
var call = Expression.Call(method, Expression.Constant("Hello, World!"));
var callLambda = Expression.Lambda<Action>(call).Compile();
callLambda(); // Output: Hello, World!
```

Member Access Expressions

- **MemberAccess:** Represents accessing a **field or property**.

```
var date = Expression.Constant(DateTime.Now);
var property = Expression.Property(date, "Year");
var memberAccessLambda = Expression.Lambda<Func<int>>(property).Compile();
```

```
Console.WriteLine(memberAccessLambda()); // Output: current year
```

Lambda Expressions

- **Lambda:** Represents a lambda expression.

```
var lambda = Expression.Lambda<Func<int, int, int>>(
    Expression.Add(param, param), param, param);
var compiledLambda = lambda.Compile();
Console.WriteLine(compiledLambda(2, 3)); // Output: 4
```

Unary Expressions

- **Convert:** Represents a type conversion operation.

```
var convert = Expression.Convert(Expression.Constant(42), typeof(double));
var convertLambda = Expression.Lambda<Func<double>>(convert).Compile();
Console.WriteLine(convertLambda()); // Output: 42.0
```

- **Negate:** Represents a numeric negation operation.

```
var negate = Expression.Negate(left);
var negateLambda = Expression.Lambda<Func<int>>(negate).Compile();
Console.WriteLine(negateLambda()); // Output: -5
```

New Expressions

- **New:** Represents calling a constructor.

```
var newExpr = Expression.New(typeof(List<int>));
var newLambda = Expression.Lambda<Func<List<int>>>(newExpr).Compile();
var list = newLambda();
Console.WriteLine(list.Count); // Output: 0
```

***Looping statement

The `for` loop is used to execute a block of code a specific number of times. Here is the general syntax:

```
for (initialization; condition; iteration)
{
    // Code to be executed
}
```

- **initialization:** Initializes the loop control variable(s).
- **condition:** The loop continues to execute as long as this condition is true.
- **iteration:** Updates the loop control variable(s) after each iteration.

Example

using System;

```
namespace ForLoopExample
{
    class Program
```

```

{
    static void Main(string[] args)
    {
        // Initialize a for loop that runs from 1 to 10
        for (int i = 1; i <= 10; i++)
        {
            // Print the current value of i
            Console.WriteLine(i);
        }
    }
}

```

Output

```

1
2
3
4
5
6
7
8
9
10

```

The `while` loop is used to repeatedly execute a block of code as long as a specified condition is true.

`while (condition)`

```

{
    // Code to be executed
}

```

Condition: The loop continues to execute as long as this condition is true. If the condition is false initially, the loop body will not be executed at all.

`using System;`

`using System;`

`namespace WhileLoopExample`

```

{
    class Program
    {
        static void Main(string[] args)
        {
            // Initialize the starting number
            int i = 1;

            // Use a while loop to print numbers from 1 to 10

```

```

while (i <= 10)
{
    // Print the current value of i
    Console.WriteLine(i);

    // Increment the counter
    i++;
}
}
}
}

```

Foreach Loop Syntax in C#

The `foreach` loop is used to iterate over the elements of a collection (like an array, list, or any enumerable object) without needing to manage the loop counter. It is particularly useful for collections where the number of elements is not known in advance.

foreach (type variable in collection)-19-7-24

```

{
    // Code to be executed for each element in the collection
}

```

- **type:** The type of the elements in the collection.
- **variable:** The loop variable that holds the current element from the collection.
- **collection:** The collection to be iterated over.

Example

```

using System;
using System.Collections.Generic;

```

```

namespace ForeachLoopExample

```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            // Create a list of strings
            List<string> names = new List<string> { "Alice", "Bob", "Charlie" };

            // Use a foreach loop to iterate over the list
            foreach (string name in names)
            {
                // Print the current name to the console
                Console.WriteLine(name);
            }
        }
    }
}

```



```

    }
}

```

*****Delegate**

- Delegates are similar to C ++ function pointer. It is referred to as type safe function pointer.
- It is type safe object that can point to one or more methods in the application.
- It holds the methods reference in an object. It calls function at runtime base on dynamic condition.
- (The delegate and method have the same signature and parameter type.)
- A delegate is a form of type-safe function pointer.
- It is used to implement callbacks and event listeners.
- It does not need to know anything about classes of methods it works with.
- It is a reference type. But instead of referring to an object, a delegate refers to a method.
- Delegates also allow methods to be passed as parameters.

Syntax

Public delegate data type (return type) delegate name (arguments / parameters)

Here, function can be assigned or defined like a variable. It can be pass the functions as the arguments.

- When we create delegates it needs the method name with its parameter or without parameter.
- The delegates check it points those methods have the same name signature and parameter type.
- It can be changed together.
- It is used to improve the performance of an application. It can involve methods synchronization.

Example

```

using System;
using System.Web.UI;

namespace DelegateExample
{
    public partial class Default : Page
    {
        // Define a delegate
        public delegate void DisplayMessage(string message);

        protected void Page_Load(object sender, EventArgs e)
        {
            if (!IsPostBack)
            {
                // Create an instance of the delegate and point it to the DisplayMessageMethod
                DisplayMessage displayMessageDelegate = new DisplayMessage(DisplayMessageMethod);
            }
        }
    }
}

```

```

        // Call the delegate
        displayMessageDelegate("Hello, this is a message from the delegate!");
    }
}

// Method that matches the delegate signature
public void DisplayMessageMethod(string message)
{
    Response.Write(message);
}
}
}

```

***C# Function

Function is a block of code that has a signature.

Function is used to execute statements specified in the code block.

A function consists of the following components:

Function name: It is a unique name that is used to make Function call.

Return type: It is used to specify the data type of function return value.

Body: It is a block that contains executable statements.

Access specifier: It is used to specify function accessibility in the application.

Parameters: It is a list of arguments that we can pass to the function during call.

C# Function Syntax

```

<access-specifier><return-type>FunctionName(<parameters>)
{
    // function body
    // return statement
}

```

C# Function: using no parameter and return type

A function that does not return any value specifies **void** type

as a return type. In the following example, a function is created without return type.

```

using System;

namespace FunctionExample

```

```

{
    class Program
    {
        // User defined function without return type
        public void Show() // No Parameter
        {
            Console.WriteLine("This is non parameterized function");
            // No return statement
        }
        // Main function, execution entry point of the program
        static void Main(string[] args)
        {
            Program program = new Program(); // Creating Object
            program.Show(); // Calling Function
        }
    }
}

```

Output:

This is non parameterized function

C# Function: using parameter but no return type

```

using System;
namespace FunctionExample
{
    class Program
    {
        // User defined function without return type
        public void Show(string message)
        {
            Console.WriteLine("Hello " + message);
            // No return statement
        }
        // Main function, execution entry point of the program
        static void Main(string[] args)
        {
            Program program = new Program(); // Creating Object

```

```

        program.Show("Rahul Kumar"); // Calling Function
    }
}
}

```

Output:

```
Hello Rahul Kumar
```

A function can have zero or any number of parameters to get data. In the following example, a function is created without parameters. A function without parameter is also known as **non-parameterized** function.

C# Function: using parameter and return type

```

using System;
namespace FunctionExample
{
    class Program
    {
        // User defined function
        public string Show(string message)
        {
            Console.WriteLine("Inside Show Function");
            return message;
        }
        // Main function, execution entry point of the program
        static void Main(string[] args)
        {
            Program program = new Program();
            string message = program.Show("Rahul Kumar");
            Console.WriteLine("Hello "+message);
        }
    }
}

```

Output:

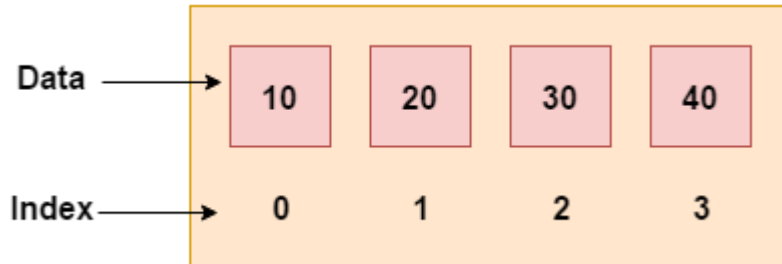
```

Inside Show Function
Hello Rahul Kumar

```

***Array

array in C# is a group of similar types of elements that have contiguous memory location. In C#, array is an *object* of base type **System.Array**. In C#, array index starts from 0. We can store only fixed set of elements in C# array.



Advantages of C# Array

- Code Optimization (less code)
- Random Access
- Easy to traverse data
- Easy to manipulate data
- Easy to sort data etc.

Disadvantages of C# Array

- Fixed size

C# Array Types

There are 3 types of arrays in C# programming:

1. Single Dimensional Array
2. Multidimensional Array
3. Jagged Array

C# Single Dimensional Array

To create single dimensional array, you need to use square brackets [] after the type.

```
int[] arr = new int[5]; //creating array
```

You cannot place square brackets after the identifier.

```
int arr[] = new int[5]; //compile time error
```

Let's see a simple example of C# array, where we are going to declare, initialize and traverse array.

```
using System;
public class ArrayExample
{
    public static void Main(string[] args)
    {
        int[] arr = new int[5]; //creating array
        arr[0] = 10; //initializing array
        arr[2] = 20;
        arr[4] = 30;

        //traversing array
        for (int i = 0; i < arr.Length; i++)
        {
            Console.WriteLine(arr[i]);
        }
    }
}
```

Output:

```
10
0
20
0
30
```

C# Array Example: Declaration and Initialization at same time

There are 3 ways to initialize array at the time of declaration.

```
int[] arr = new int[5]{ 10, 20, 30, 40, 50 };
```

We can omit the size of array.

```
int[] arr = new int[] { 10, 20, 30, 40, 50 };
```

We can omit the new operator also.

```
int[] arr = { 10, 20, 30, 40, 50 };
```

Let's see the example of array where we are declaring and initializing array at the same time.

```
using System;
public class ArrayExample
{
    public static void Main(string[] args)
    {
        int[] arr = { 10, 20, 30, 40, 50 };//Declaration and Initialization of array

        //traversing array
        for (int i = 0; i < arr.Length; i++)
        {
            Console.WriteLine(arr[i]);
        }
    }
}
```

Output:

```
10
20
30
40
50
```

C# Array Example: Traversal using foreach loop

We can also traverse the array elements using foreach loop. It returns array element one by one.

```
using System;
public class ArrayExample
{
    public static void Main(string[] args)
    {
        int[] arr = { 10, 20, 30, 40, 50 };//creating and initializing array

        //traversing array
        foreach (int i in arr)
        {
```

```
        Console.WriteLine(i);  
    }  
}  
}
```

Output:

```
10  
20  
30  
40  
50
```